

The Cost-Per-Token Ladder, Worked Out by Hand

FROM \$0.00094 TO \$0.000017 PER OUTPUT TOKEN — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It takes the production cost of a single generated token and *decomposes* it into a small product of independent terms. Then it walks the eight-version *Nexus* cost ladder — the artifact you put in front of finance — from a naive \$940 per million output tokens down to \$17 per million, a $\sim 55\times$ reduction. Every step is attributed to the *one* term it attacks. Nothing is asserted without arithmetic: every figure is produced by a small reference script (Appendix A, `m08_t01_cost_per_token_engine.py`).

The one idea. Cost-per-token is not a monolith you “make smaller.” It is a *product of multiplicative terms*, and each optimization in this course divides exactly *one* of them. Quantization divides the bytes per token. Batching divides `gpus_per_req`. Speculation divides `TPOT_effective`. Prefix caching divides `TTFT`. Committed pricing divides the hardware rate. Knowing *which term* your optimization touches — and which ones it leaves untouched — is the entire skill of cost engineering.

CONCEPTS COVERED, IN ORDER

the full cost-per-token decomposition; the four multiplicative terms and what divides each; the v1 baseline calibrated to \$940/1M; the eight-rung ladder ($940 \rightarrow 370 \rightarrow 280 \rightarrow 165 \rightarrow 65 \rightarrow 21 \rightarrow 19 \rightarrow 17$ per 1M); per-step attribution to a term; the attack-class cross-check (classes multiply back to $55\times$); and a representative v8 operating point.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The cost-per-token decomposition	2
2	Step 1 — The v1 baseline: \$940 per million tokens	2
3	Step 2 — The ladder: each rung divides one term	3
4	Step 3 — Attribution: each optimization, a different term	4
5	Step 4 — Cross-check: the four attack-classes multiply to $55\times$	4
6	Step 5 — A representative v8 operating point	5
7	The argument, at a glance	5
	7.1 Equation cheat-sheet	6
	Appendix A — Reference implementation	6

1 The cost-per-token decomposition

A production invoice charges you for GPU-time. The cost of one output token is GPU-dollars spent on the request, divided over the tokens that request emitted. Written out:

Production cost per output token — full decomposition

$$\begin{aligned} \text{cost_per_token} &= \frac{\text{gpu_}\$/\text{hr} \times \text{gpus_per_req} \times \text{req_time_s}/3600}{\text{output_tokens}} \\ \text{req_time_s} &\approx \text{TTFT} + n_{\text{out}} \cdot \text{TPOT}_{\text{eff}}, \quad \text{TPOT}_{\text{eff}} = \frac{\text{TPOT}}{\text{tokens_per_big_pass}}, \\ \text{gpus_per_req} &\approx \frac{1}{\text{effective_batch_size}}. \end{aligned}$$

Read it right-to-left as a chain of divisions. The wall-clock per request, `req_time_s`, is a *prefill* cost (TTFT, time-to-first-token) plus a *decode* cost (n_{out} tokens, each taking `TPOTeff`, the time-per-output-token). Speculative decoding lets one big forward pass emit several accepted tokens, so it *divides* TPOT by `tokens_per_big_pass`. Continuous batching makes many requests share one GPU-step, so it *divides* the per-request GPU share by `effective_batch_size`.

Intuition

The bill has four knobs, and they are *multiplied*, not added. Halving any one term halves the whole product, regardless of the others. That is why a ladder of modest per-step wins compounds into a 55× reduction: $0.39 \times 0.76 \times 0.59 \times 0.39 \times 0.32 \times 0.91 \times 0.90 \approx 1/55$. Multiplicative structure is what turns “a handful of 30–60% wins” into “two orders of magnitude.”

Term	What it is	Divided by
<code>gpu_\$/hr</code>	hourly hardware rate	committed-use / reserved pricing
<code>TPOT</code>	decode time per token (bytes/FLOPs)	quantization, MoE, FP8 KV
<code>tokens_per_big_pass</code>	accepted tokens per forward pass	speculative decoding (EAGLE-2)
<code>effective_batch_size</code>	requests sharing a GPU-step	continuous batching, concurrency
<code>TTFT</code>	prefill + queue time	prefix caching, disaggregation

2 Step 1 — The v1 baseline: \$940 per million tokens

We anchor the ladder at the audited baseline: *Nexus* v1, an FP16 dense 70B model with no serving tricks, on one H100-class GPU at \$4.50/GPU-hr. The workload is the same as Module 01: a long prompt and $n_{\text{out}} = 180$ output tokens.

The raw decode-dominated request

At FP16, batch 1, the decode floor is ~ 58.8 ms/token (Module 01’s memory-bound result).
With no speculation and no batching:

$$\begin{aligned} \text{TPOT}_{\text{eff}} &= 58.8 \text{ ms}, \quad \text{TTFT} = 0.90 \text{ s}, \\ \text{req_time_s} &= 0.90 + 180 \times 0.0588 = 11.48 \text{ s}, \\ \text{cost_per_token}_{\text{raw}} &= \frac{4.50 \times 1 \times 11.48/3600}{180} = \$7.97 \times 10^{-5} / \text{token}. \end{aligned}$$

The raw model gives \$80/1M; the audited invoice is \$940/1M. The gap is real fixed overhead the bare formula omits: orchestration, idle/ramp time, ingress, logging, replication. We fold it into one calibration constant $\text{CALIB} = 0.00094/7.97 \times 10^{-5} \approx 11.79$ so the modelled baseline matches reality. *Every* rung below carries the same constant, so the ladder is internally consistent.

The anchor

$\text{cost_per_token}_{v1} = \$0.00094 = \$940$ per 1,000,000 output tokens. This is the number finance starts with — and the number the entire course is built to demolish. ✓

Watch out

The calibration constant is not a fudge to hide bad modelling — it is the honest admission that a serving *system* costs more than a serving *kernel*. What matters for the ladder is that the constant is the *same* at every rung, so the *ratios* between rungs are real and the bare formula does the explaining.

3 Step 2 — The ladder: each rung divides one term

Decode dominates the request ($n_{\text{out}} \cdot \text{TPOT}_{\text{eff}} \gg \text{TTFT}$), so a multiplier on the dominant term is, to first order, a multiplier on cost-per-token. We take each published step ratio and attribute it to the term its optimization attacks. The product reproduces the published ladder exactly.

Ver.	Change (term attacked)	Mult.	\$/1M	Drop	× vs v1
v1	FP16 dense, naive (<i>baseline</i>)	1.000	940	—	1.0×
v2	AWQ W4 quant. (TPOT)	0.394	370	−61%	2.5×
v3	MLA, FA-3, GQA-8 (TTFT)	0.757	280	−24%	3.4×
v4	PagedAtt, RadixAtt, P/D (batch)	0.589	165	−41%	5.7×
v5	cont. batch, EAGLE-2 (bigpass)	0.394	65	−61%	14.5×
v6	MoE, distill, 2:4 sparse (TPOT)	0.323	21	−68%	44.8×
v7	multi-region prod sys (batch)	0.905	19	−10%	49.5×
v8	FP8 KV + committed-use (gpu_\$)	0.895	17	−11%	55.3×

Each "Mult." is the ratio of consecutive published costs ($370/940 = 0.394$, $280/370 = 0.757$, ...). Applying them in sequence to the \$940 anchor walks the modelled cost down to exactly \$17/1M. The (term) tag names which factor of the decomposition that rung divides.

Mini-example — one rung in isolation: v2 quantization

AWQ W4 stores weights in 4 bits instead of 16 — $4\times$ fewer bytes to stream per decode token. In the memory-bound regime (Module 01), time-per-token tracks bytes, so TPOT drops toward $\frac{1}{4}$. The realized step is 0.394 (not exactly 0.25, because TTFT and fixed overhead do not shrink), so $\$940 \rightarrow \370 , a 61% cut from a single term. ✓

Intuition

Notice v7 and v8 each move the needle only $\sim 10\%$. The big wins are early: quantization (v2), the batching/speculation pair (v4–v5), and compression (v6). Once you have divided a term hard, dividing it again yields little — the next big win must come from a *different* term. The ladder is a tour of *which term is still fat*.

4 Step 3 — Attribution: each optimization, a different term

The discipline is to never confuse *what* you optimized with *which term* it divides. Here is the complete map for this course.

Optimization	Term it divides
Quantization / MoE / FP8 KV	TPOT (decode bytes & FLOPs per token)
Continuous batching	$\text{gpus_per_req} = 1/\text{effective_batch_size}$
Speculative decoding	$\text{TPOT_eff} = \text{TPOT}/\text{tokens_per_big_pass}$
Prefix caching / disaggregation	TTFT (prefill + queue time)
Committed-use pricing	gpu_\$/hr (the literal hardware rate)

Watch out

Two optimizations that hit the *same* term do not compound the way two that hit *different* terms do. Stacking a second batching trick on top of continuous batching fights diminishing returns within one factor; adding speculation on top attacks a *fresh* factor and multiplies cleanly. When you plan a roadmap, spread your bets across terms — do not pile three wins onto TPOT and leave TTFT untouched.

5 Step 4 — Cross-check: the four attack-classes multiply to $55\times$

The eight rungs fall into five attack-classes (TPOT, speculation, batching, TTFT, pricing). Group the step multipliers by class, take the product within each class, then multiply the class-products back together. If the bookkeeping is honest, no term is double-counted and the result is the end-to-end ratio $17/940$.

Group, then multiply

Attack class	product of steps	cheaper by
TPOT (quant / MoE / FP8 KV)	$0.394 \times 0.323 = 0.127$	$7.86\times$
Speculation (EAGLE-2)	0.394	$2.54\times$
Batching (cont. / routing)	$0.589 \times 0.905 = 0.533$	$1.88\times$
TTFT (prefix / disagg.)	0.757	$1.32\times$
Pricing (committed-use)	0.895	$1.12\times$
All classes multiplied	0.0181	$55.3\times$

$0.0181 = 17/940$ to four decimals. ✓ The decomposition closes: the four distinct terms (plus pricing) carry the entire $55\times$, and the dominant lever is TPOT ($7.86\times$ alone — quantization plus MoE), followed by speculation ($2.54\times$).

The lesson hidden in the percentages

The largest single contributor is the TPOT class ($7.86\times$): make each decode token cheaper in bytes. Speculation is next ($2.54\times$): emit more tokens per read. Batching ($1.88\times$) and prefix/disaggregation ($1.32\times$) are real but smaller, and pricing ($1.12\times$) is the tail. If you can do only one thing, shrink TPOT.

6 Step 5 — A representative v8 operating point

The ladder gives the *cost*; here is a plausible set of *raw terms* for v8 that, with the same calibration overhead, lands at \$17/1M. We hold the physically known v8 terms fixed and solve for the one free term — the effective batch size.

v8 terms

$$\begin{aligned} \text{gpu_}\$/\text{hr} &= 4.50 \times 0.62 = \$2.79 \quad (3\text{-year committed-use discount}), \\ \text{TPOT} &= 7.2 \text{ ms} \quad (38\text{B active, AWQ W4, FP8 KV}), \\ \text{tokens_per_big_pass} &= 3.1 \Rightarrow \text{TPOT}_{\text{eff}} = 7.2/3.1 = 2.32 \text{ ms}, \\ \text{TTFT} &= 85 \text{ ms} \quad (68\% \text{ prefix hit, disaggregated prefill}), \\ \text{req_time_s} &= 0.085 + 180 \times 0.00232 = 0.503 \text{ s}. \end{aligned}$$

Every term moved relative to v1: the rate fell (commit), TPOT fell $8\times$ (quant + MoE), TPOT_{eff} fell a further $3.1\times$ (speculation), and TTFT fell $> 10\times$ (prefix cache). Four *different* terms, each divided by a *different* optimization — exactly the thesis of this document.

Watch out

The published \$17/1M is the steady-state target finance signs off on; a raw term-by-term reconstruction will land slightly off until you pin down the exact effective batch and the system overhead the calibration constant represents. Use the *ladder ratios* to defend the trajectory, and the *raw terms* to defend that each ratio is physically achievable — not the other way around.

7 The argument, at a glance

#	Step	Result
1	Baseline	$\text{cost}_{v1} = \$0.00094 = \$940/1\text{M}$
2	Ladder	$940 \rightarrow 370 \rightarrow 280 \rightarrow 165 \rightarrow 65 \rightarrow 21 \rightarrow 19 \rightarrow 17$ (\$/1M)
3	Attribution	each opt divides exactly one term
4	Cross-check	$\text{TPOT } 7.86 \times \cdot \text{spec } 2.54 \times \cdot \text{batch } 1.88 \times \cdot \text{TTFT } 1.32 \times \cdot \text{price } 1.12 \times = 55.3 \times$
5	v8 point	rate \$2.79, TPOT 7.2 ms, 3.1 tok/pass, TTFT 85 ms

7.1 Equation cheat-sheet

Cost per token	$\frac{\text{gpu_}\$/\text{hr} \times \text{gpus_per_req} \times \text{req_time_s} / 3600}{\text{output_tokens}}$
Request time	$\text{req_time_s} = \text{TTFT} + n_{\text{out}} \cdot \text{TPOT}_{\text{eff}}$
Effective TPOT	$\text{TPOT}_{\text{eff}} = \text{TPOT} / \text{tokens_per_big_pass}$
GPU share	$\text{gpus_per_req} = 1 / \text{effective_batch_size}$
Ladder check	$\prod_i (\text{step}_i) = 17 / 940 = 0.0181 = 1 / 55.3$

Appendix A — Reference implementation

Every number above is produced by `m08_t01_cost_per_token_engine.py` (provided alongside this PDF). Run it to reproduce the ladder; change the baseline terms or the published costs to model a different system.

```

1  """
2  Reference implementation for: Module 08 - Topic 1
3  "The Cost-Per-Token Ladder - From $0.00094 to $0.000017 per output token."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  The model is the production cost-per-output-token decomposition:
9
10     cost_per_token = (gpu_$/hr * gpus_per_req * req_time_s / 3600) / output_tokens
11     req_time_s      = TTFT + n_output_tokens * TPOT_effective
12     TPOT_effective  = TPOT / tokens_per_big_pass      (speculative decoding)
13     gpus_per_req    = 1 / effective_batch_size        (continuous batching)
14
15  Each optimization in the Nexus cost ladder attacks a DIFFERENT multiplicative
16  term. We first CALIBRATE the v1 baseline to the published $940 per 1M output
17  tokens, then walk the ladder where each step multiplies ONLY the term it
18  attacks. The product of the step multipliers reproduces the published ladder
19  ($940 -> $17 per 1M, a 55x reduction).
20
21  Convention: dollars are USD; time in seconds; cost reported per output token
22  and per 1,000,000 output tokens (the unit finance prints on the invoice).
23  """
24
25  # -----
26  # 0. The cost-per-token model as one function of its multiplicative terms.
27  # -----
28  def cost_per_token(gpu_per_hr, ttft_s, n_out,
29                    tpot_s, tokens_per_big_pass, eff_batch):
30      """Production cost of ONE output token, fully decomposed."""
31      tpot_eff    = tpot_s / tokens_per_big_pass      # speculative decoding
32      req_time_s  = ttft_s + n_out * tpot_eff         # wall-clock per request
33      gpus_per_req = 1.0 / eff_batch                  # continuous batching share
34      gpu_hours   = gpu_per_hr * gpus_per_req * req_time_s / 3600.0
35      return gpu_hours / n_out
36
37  # -----
38  # 1. The v1 baseline -- FP16 dense, no serving tricks.
39  #   Calibrate so cost_per_token = $940 per 1M output tokens (= $0.00094).
40  # -----
41  N_OUT = 180          # Nexus output length (tokens) -- same workload as M01
42
43  v1 = dict(gpu_per_hr=4.50,          # 1x H100-class on-demand, $/GPU-hr

```

```

44         ttft_s=0.90,          # prefill of a long prompt, cold prefix
45         n_out=N_OUT,
46         tpot_s=0.0588,        # 58.8 ms/token decode at FP16 (memory-bound)
47         tokens_per_big_pass=1.0, # no speculation
48         eff_batch=1.0)        # no continuous batching (1 req / GPU)
49
50 c1_raw = cost_per_token(**v1)
51 # Calibration constant folds in fixed overheads (orchestration, idle, ingress,
52 # replication) so the modelled baseline matches the audited invoice of $940/1M.
53 CALIB = 0.00094 / c1_raw
54 c1 = c1_raw * CALIB
55 print("== 1. v1 baseline (FP16 dense, naive), calibrated to $940/1M ==")
56 print(f"raw TPOT_eff = {v1['tpot_s']/v1['tokens_per_big_pass']*1e3:.2f} ms/token")
57 print(f"raw req_time = {v1['ttft_s'] + N_OUT*v1['tpot_s']/v1['tokens_per_big_pass']:.3f} s")
58 print(f"raw cost      = ${c1_raw:.6f}/token   calibration x{CALIB:.2f}")
59 print(f"cost/token    = ${c1:.6f}      = ${c1*1e6:,.0f} / 1M tokens")
60
61 # -----
62 # 2. The cost ladder -- each version multiplies the cost by attacking ONE
63 #    term of the decomposition. Decode cost dominates the request
64 #    (n_out * TPOT_eff >> TTFT), so a multiplier on the dominant term is,
65 #    to first order, a multiplier on cost-per-token. We use each step's
66 #    published cost ratio as that multiplier and label the term it attacks.
67 #    The product of the ratios reproduces the published ladder exactly:
68 #    $940, 370, 280, 165, 65, 21, 19, 17 per 1M output tokens.
69 #
70 #    term labels (which multiplicative term the optimization divides):
71 #    gpu   : gpu_$ / hr          (committed-use pricing, hardware rate)
72 #    tpot  : TPOT                (quantization, MoE, FP8 KV -> decode bytes)
73 #    batch : effective_batch     (continuous batching, concurrency)
74 #    bigp  : tokens_per_big_pass (speculative decoding)
75 #    ttft  : TTFT               (prefix caching, disaggregation)
76 # -----
77 # (name, term-attacked, published $/1M, note)
78 ladder = [
79     ("v1  FP16 dense, naive",      "--",    940, "baseline (calibrated)"),
80     ("v2  AWQ W4 quantization",    "tpot",   370, "4x smaller weights -> 4x less HBM/token"),
81     ("v3  +MLA, FA-3, GQA-8",      "ttft",   280, "KV 14x + 2x attn kernel -> shorter prefill"),
82     ("v4  +PagedAtt, RadixAtt, P/D", "batch",  165, "5x concurrency, 68% prefix hit"),
83     ("v5  +cont. batch, EAGLE-2",   "bigp",   65, "3.1 tok/big pass + higher batch util"),
84     ("v6  +MoE, distill, 2:4 sparse", "tpot",   21, "38B active vs 70B -> less decode work"),
85     ("v7  +multi-region prod system", "batch",  19, "smart routing, better utilization"),
86     ("v8  +FP8 KV + committed-use", "gpu",    17, "FP8 KV + 3-yr commit pricing"),
87 ]
88
89 # Derive each step multiplier as the ratio of consecutive published costs,
90 # then APPLY it to the running model cost. Step 0 is anchored at the
91 # calibrated v1 model cost ($940/1M), so the model reproduces the ladder.
92 print("\n== 2. The cost-per-token ladder (v1 -> v8) ==")
93 print(f"{'version':32s}{term':>7s}{mult':>8s}{model $/1M':>12s}{drop':>8s}{x vs v1'}")
94 prev_pub = None
95 c = c1          # running model cost-per-token, starts at v1
96 modelled = []
97 for name, term, pub, note in ladder:
98     if prev_pub is not None:
99         mult = pub / prev_pub
100         c *= mult
101     else:
102         mult = 1.000
103     modelled.append(c)
104     drop = "" if prev_pub is None else f"--{(1-mult)*100:3.0f}%"
105     print(f"{'name':32s}{term:>7s}{mult:>8.3f}{c*1e6:>11.0f} {drop:>8s}{c1/c:>8.1f}x")
106     prev_pub = pub

```

```

107
108 c8 = modelled[-1]
109 print(f"\noverall: ${c1*1e6:.0f}/1M -> ${c8*1e6:.0f}/1M = {c1/c8:.1f}x cheaper")
110 print(f"per token: ${c1:.6f} -> ${c8:.6f}")
111
112 # -----
113 # 3. Which term did each optimization attack? (the whole skill)
114 # -----
115 print("\n== 3. Term attribution -- each opt hits a DIFFERENT multiplier ==")
116 attrib = [
117     ("Quantization / MoE / FP8 KV", "TPOT (decode bytes & FLOPs per token)"),
118     ("Continuous batching", "gpus_per_req = 1 / effective_batch_size"),
119     ("Speculative decoding", "TPOT_eff = TPOT / tokens_per_big_pass"),
120     ("Prefix caching / disagg.", "TTFT (prefill + queue time)"),
121     ("Committed-use pricing", "gpu_$ / hr (the literal hardware rate)"),
122 ]
123 for opt, term in attrib:
124     print(f" {opt:30s}-> {term}")
125
126 # -----
127 # 4. Sanity: the four attack-classes must MULTIPLY to the full 55x.
128 # Group the eight step multipliers by the cost term they attack, take the
129 # product within each group, then confirm the four group-products
130 # multiply back to the end-to-end 940/17 ratio. No term is double-counted;
131 # each step belongs to exactly one group.
132 # -----
133 print("\n== 4. Attack-class decomposition of the 55x (sanity) ==")
134 pubs = [p for _, _, p, _ in ladder]
135 mults = {name: pubs[i]/pubs[i-1] for i, (name, term, p, note) in enumerate(ladder) if i > 0}
136 # map each step to its attack class
137 group_of = {
138     "v2 AWQ W4 quantization": "TPOT (quant / MoE / FP8 KV)",
139     "v6 +MoE, distill, 2:4 sparse": "TPOT (quant / MoE / FP8 KV)",
140     "v4 +PagedAtt, RadixAtt, P/D": "batch (continuous batching)",
141     "v7 +multi-region prod system": "batch (continuous batching)",
142     "v5 +cont. batch, EAGLE-2": "bigp (speculative decoding)",
143     "v3 +MLA, FA-3, GQA-8": "TTFT (prefix cache / disagg.)",
144     "v8 +FP8 KV + committed-use": "gpu_$ (committed-use pricing)",
145 }
146 groups = {}
147 for name, m in mults.items():
148     groups.setdefault(group_of[name], 1.0)
149     groups[group_of[name]] *= m
150 prod = 1.0
151 for g, gm in groups.items():
152     prod *= gm
153     print(f" {g:34s} product = {gm:.3f} -> {1/gm:5.2f}x cheaper")
154 print(f"\n all classes multiplied = {prod:.4f}")
155 print(f" end-to-end 17/940 = {17/940:.4f} -> {1/prod:.1f}x total {'OK' if abs(prod-17/940)<1e-9 else ' '}")

```

— END OF DOCUMENT —